

1. Load

We started off our approach by simply loading the datasets into our project.

2. Missing-value handling

#

We handled missing data values by using per-station linear interpolation for
short gaps (≤ 6 h), then per-station median fill for anything longer as to
ensure accurate estimations.

For the wind directions, we used the mode for that particular station
(using North if no mode exists).

#

By doing this, we filled 39323 missing values from the training set and 4226
from the test set. We also noted 1526 cases of extreme weather events where
there were more than 500 PM10 particles.

3. Feature Engineering

The main features we implemented were finding the wind vectors (factoring
both direction and speed), using sine/cosine transformations on the dates
(to account for their repeating nature), accounting dewpoint depression
(since a smaller difference between temperature and dewpoint promotes
particle growth), finding rolling averages, calculating lag, and calculating
historical averages for each hour of day (only using past data to prevent
leakages)

#

NOTE: earlier versions added "pm25_actual"-derived features reconstructed
by shifting the target column (since there's no raw PM2_5 reading in this
dataset). Verified against the real data: this made local validation RMSE
look great (~ 17.6) but only because the feature is fully available in
train/val — it's genuinely unknown for $\sim 99\%$ of real test rows. Masking it
out to match real test conditions pushed validation RMSE to ~ 66.3 , meaning
the leaderboard score would have badly missed the CV estimate. Removed
entirely here so validation RMSE reflects what the model can actually see
on the leaderboard test set.

4. Combine & Build Features

Here, we combined the cleaned train and test datasets into one, then passed
the whole set into the feature-building function from the previous phase.
This was done since it makes it more efficient to add features on both sets
simultaneously than to do it separately.

5. Separate Sets

We now separated the training and test sets again for further model training
now that they had engineered features.

6. Features & Categorical Handling

In this phase, we prepared our predictor variables based on the feature data.
We specifically checked for distinct categorical features (such as the station)
and saved the non-categorical strings (being text flags) as boolean variables.
In total we retained 157 total features.

7. Time-based Validation Split

Our validation strategy was to separate the training data into an 80/20
temporal split, meaning the first 80% of the period (288,762 rows) was used
for training and the last 20% (72,192 rows) was used for validation. We then
separated the predictor (engineered features) and target columns (measured
PM2.5 particles) for both training and validation sets.

8. Hyperparameter Search (early stopping)

To find an ideal model, we used grid search hyperparameter optimization
for a LightGBM regressor. We chose this model first because it is well suited
for large datasets and categorical data.

This process tested out different combinations of model complexity/number of
leaves and the learning rate, picking the one with the lowest RMSE.

To prevent overfitting and long computing times, we set 'early stopping',
meaning that if 50 consecutive iterations failed to set the lowest RMSE, then
the phase would end.

The best parameters found were about 31 leaves, a learning_rate of 0.05,
751 estimators, leading to a RMSE of about 24.8430.

9. Train final LightGBM

We now train the LightGBM model obtained from the previous phase by testing
it on the sets found in phase 7. Here, we use the ideal hyperparameters found
in phase 8 to create an accurate fit to the training sets, before testing it
on the validation sets.

From this, we obtain a MAE of about 13.4656, a median absolute error of about
7.9119, and a 90th percentile absolute error of about 29.8258.

10. Train XGBoost

We also trained a XGBoost regression model, which differentiates itself from
LightGBM due to it relying on building trees level by level instead of leaf by leaf,
resulting in more balanced and thus stable trees. We again used early stopping,
with 800 estimators/trees, a learning rate of 0.03, histogram-based binning,
and a tree depth of up to 6 levels to prevent overfitting. We then treated it

on the sets, finding a RMSE of about 25.0974 with a model of 799 trees.

11. Blend weight

Now that two models had been tested, we now had to select the optimal weighted
average between the LightGBM and XGBoost models such that validation errors
were minimised. This was done by testing different weights on LightGBM in
increments of 0.1, then finding the weight with the smallest RMSE. In this
case, 0.8 produced the smallest RMSE at about 24.8185. Following this, we also
took note on the most important features of the model to help us make further
tweaks.

12. Retrain on ALL data, predict on real test set

Using the ideal weight, models, and hyperparameters found in previous phases,
we retrained our model off of all the training data, including both the training
and validation sets. We then used the resulting model on the real test set
to create finalised predictions.

13. Export Submission

Finally, we exported our submission and submitted it.